Chapter 2
Neural Networks

Part 3

Deep Learning

Dr. Charles Yaacoub

Regularization

- Usually, large weights may lead to overfitting

  (e.g. $x_1+x_2=0$ vs. $20x_1+20x_2=0$, the first is preferred)

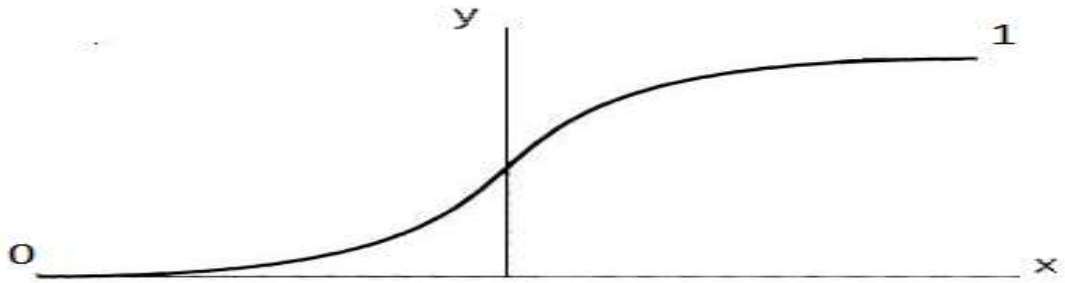
- Solution: penalize large weights
  - new loss = old loss + weight penalty
  - L1 regularization: weight penalty = $\lambda(|w_1|+\ldots+|w_n|)$
  - L2 regularization: weight penalty = $\lambda(w_1^2+\ldots+ w_n^2)$
  - Other regularizations…

Dropout

- During training, some coefficients might get too large while others too small
  - Large coefficients dominate the training
  - Some parts of the network do not train

- Solution: dropout
  - A form of regularization
  - On each epoch, randomly disable some nodes (simply, do not use these nodes during that pass)
  - Reduces the chances of overfitting (as does regularization, in general)

Vanishing Gradient



$$\frac{\partial E}{\partial W_{11}^{(1)}} = \underbrace{\frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial h_1} \frac{\partial h_1}{\partial W_{11}^{(1)}}}_{\text{Each: very small value}}$$

- Sigmoid function σ(x) is almost flat for most values of x.
  - Derivative is too small (close to zero)
  - In gradient descent: product of such values = smaller values
  - Weights update: negligible (network not learning or learning very slowly)

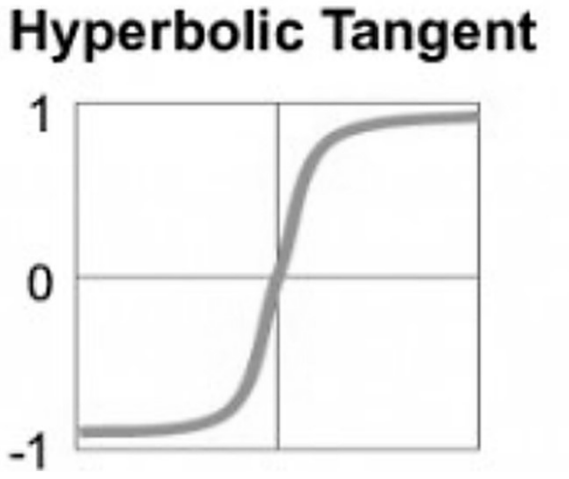
- Solution: consider other activation functions

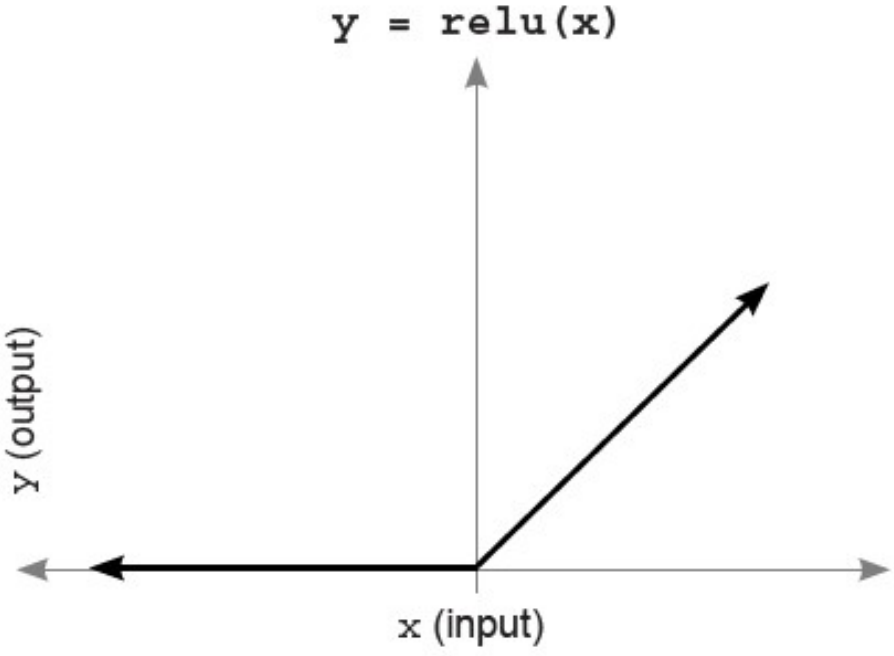
Activation Functions

- So far:
  - Step
  - Sigmoid
  - Softmax (for output layer)

- Now:
  - Tanh: similar to sigmoid, output between -1 and +1
  - ReLU: max(0, input)
    - **Re**ctified **L**inear **U**nit
    - Negative output clipped to 0
    - Derivative is 1 (positive nbrs)

- Later: others



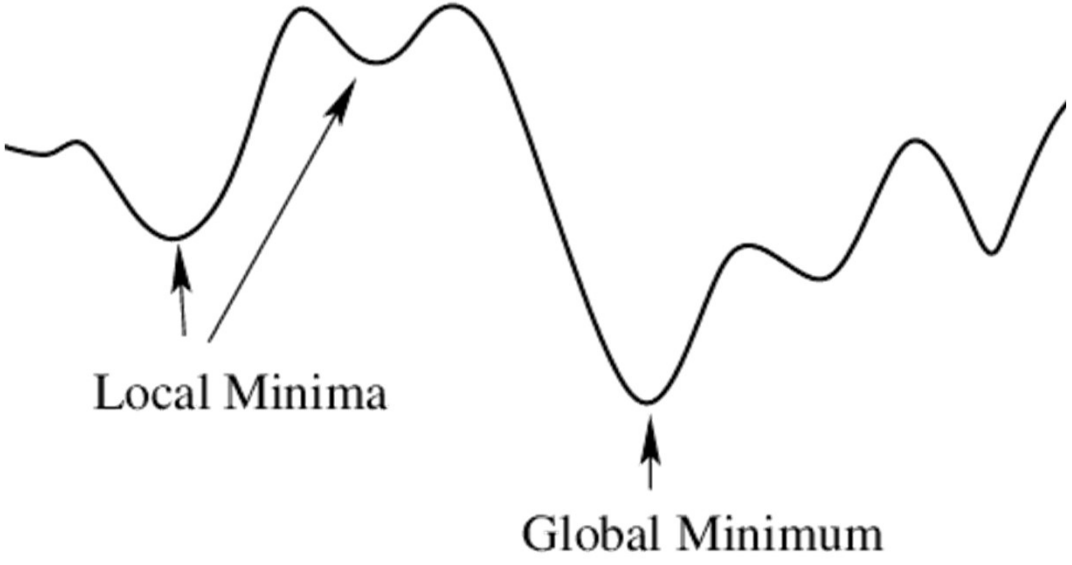


Stochastic Gradient Descent

- In gradient descent, we consider ALL the training data to update the weights ONCE (in an EPOCH)
- Instead:
  - Split the training data into subsets called **BATCHES**
  - In each epoch, iterate over the batches and update the weights after each batch
- For example:
  - Training data: 1000 records split in 10 batches of 100 records
  - In each epoch
    - use the 1000 records and update the weights once
      V/S
    - use the batches and update the weights 10 times per epoch

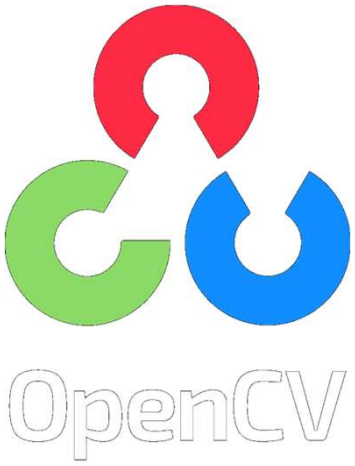
Momentum

- For each update step, include a fraction (a scaled version) of previous steps
- Step(n) = Step(n) + β Step(n-1) + β^2 Step(n-2) + …
- $0 \leq \beta \leq 1$
- This helps the gradient descent avoid falling into local minima while finding the set of weights that leads to the global minimum (of the loss function)

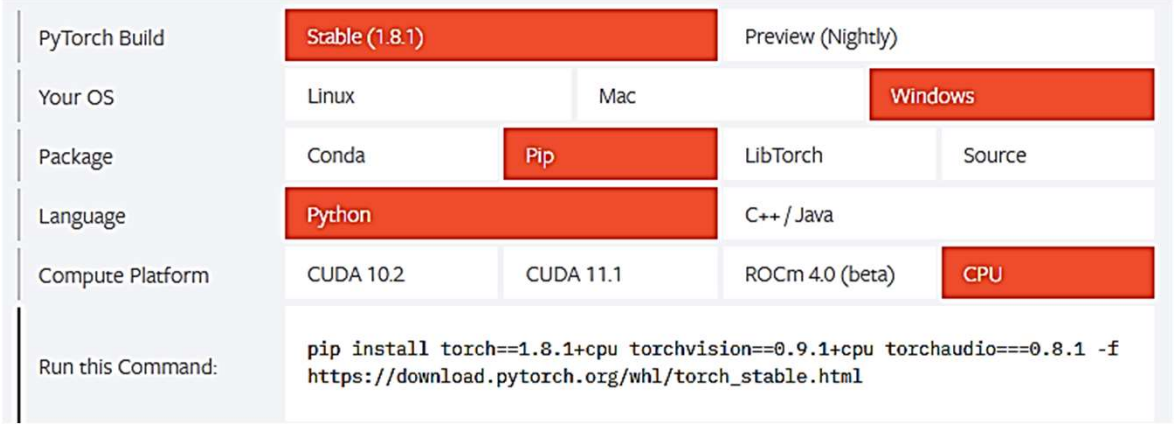


Preparing for next chapter

- Install OpenCV
  - A library for image processing and computer vision
  - In a shell prompt, run: **pip install opencv-python**



- Install PyTorch
  - An open source machine learning framework
  - Go to https://pytorch.org/
  - Scroll to reach the following:



    select you OS (e.g. Windows) & package installer (e.g. Pip)

    if you have a GPU, choose a compatible compute platform
    CUDA (for NVIDIA) or ROCm (for AMD); make sure to verify your GPU is compatible with the chosen platform
    if you do not have a GPU, select CPU
  - Copy the displayed command and run it in your shell prompt